

Интерпретатор Mini-Lisp на Haskell

1 Формальная постановка задачи

1.1 Синтаксис языка

```
<expr>      ::= <number> | <atom> | <list> | <quoted>

<number>    ::= '-'? <digit>+
<digit>     ::= '0'..'9'

<atom>      ::= <letter> <atom-rest>*
<atom-rest> ::= <letter> | <digit> | '-' | '_' | '?' | '!'
<letter>    ::= 'a'..'z' | 'A'..'Z' | '+' | '-' | '*' | '/' | '=' | '<' | '>'

<list>      ::= '(' <expr>* ')'          (* обычный список *)
              | '(' <expr> '.' <expr> ')' (* точечная пара *)

<quoted>    ::= '\' <expr>

<expr>      ::= <number> | <atom> | <list> | <quoted>
```

1.2 Семантика

1.2.1 Типы данных

- Atom String — атом (символьное имя)
- Number Integer — целое число
- List [Val] — обычный список
- DottedPair Val Val — точечная пара

1.2.2 Встроенные константы

- t — логическая истина
- nil или () — пустой список (логическая ложь)

1.2.3 Специальные формы

- (quote <expr>) — возвращает выражение без вычисления
- (if <cond> <then> <else>) — условное выражение
- (define <var> <expr>) — определение переменной

1.2.4 Встроенные функции

- (car <list>) — первый элемент списка или точечной пары
- (cdr <list>) — хвост списка или второй элемент пары
- (cons <x> <y>) — конструирование пары или списка
- (eq <x1> <x2>) — сравнение на равенство
- (+ <x1> <x2>), (- <x1> <x2>), (* <x1> <x2>), (/ <x1> <x2>) — арифметические операции
- (< <x1> <x2>), (> <x1> <x2>), (= <x1> <x2>) — сравнение чисел

2 Описание алгоритмов

2.1 Лексический анализ и парсинг

Для разбора используется библиотека `Parsec` — реализация комбинаторных парсеров.

2.1.1 Алгоритм парсинга чисел

1. Опционально обрабатываем знак минуса (только если после него идёт цифра)
2. Парсим одну или более цифр
3. Преобразуем строку в целое число
4. Применяем знак (если был)

2.1.2 Алгоритм парсинга атомов

1. Первый символ: буква или один из символов `+ - * / = < >`
2. Последующие символы: буквы, цифры или `- _ ? !`
3. Формируем атом с полученным именем

2.1.3 Алгоритм парсинга списков

Парсер использует стратегию с возвратом (`try`):

1. Пробуем распознать пустой список `()`
2. Если не пустой — пробуем распознать точечную пару `(a . b)`
3. Иначе — распознаём обычный список `(a b c ...)`

Для обычного списка:

1. Читаем открывающую скобку
2. Парсим первое выражение
3. Повторяем парсинг следующих выражений (ноль или более)
4. Читаем закрывающую скобку

2.1.4 Алгоритм парсинга quoted-выражений

1. Распознаём символ ' ,
2. Парсим следующее выражение
3. Преобразуем в (quote <expr>)

2.2 Интерпретация

2.2.1 Управление переменными

- **getVar** — линейный поиск переменной в списке окружения (ассоциативном списке)
- **setVar** — добавление новой пары (имя, значение) в начало окружения (реализует динамическую область видимости)
- Запрещено переопределение встроенных констант `t`, `nil`, `T`, `NIL`

2.2.2 Приведение к логическому значению

Функция `toBool`:

- Пустой список (`List []`) $\rightarrow$ `False`
- Атом `"nil"` $\rightarrow$ `False`
- Атом `"t"` $\rightarrow$ `True`
- Любое другое значение $\rightarrow$ `False`

2.2.3 Алгоритм eval (вычисление выражения)

Функция `eval :: Variables -> Val -> Either String (Val, Variables)`:

1. **Число** — возвращается без изменений
2. **Атом (переменная)** — ищется в окружении; если не найдена — ошибка
3. **Список** — анализируется первый элемент:
 - `quote` — возвращает второй аргумент без вычисления
 - `if` — вычисляется условие; в зависимости от истинности вычисляется одна из ветвей
 - `define` — вычисляется выражение, переменная добавляется в окружение
 - `car`, `cdr`, `cons`, `eq` — вычисляются аргументы, затем выполняется операция
 - `+`, `-`, `*`, `/`, `<`, `>`, `=` — вычисляются оба аргумента (с последовательным обновлением окружения), преобразуются к числам, выполняется операция
 - *иначе* — ошибка (в данной реализации функции не могут быть определены пользователем)
4. **Пустой список** — возвращается пустой список

2.2.4 Алгоритм работы с точечными парами

- `car` от `DottedPair x y` $\rightarrow x$
- `cdr` от `DottedPair x y` $\rightarrow y$
- `cons`:
 - Если второй аргумент — `List ys`, результат — `List (x:ys)`
 - Иначе — `DottedPair x y`

2.3 REPL

Алгоритм работы REPL:

1. Вывод `lisp>`
2. Чтение строки ввода
3. Проверка на команду `exit`
4. Парсинг строки в `Val`
5. Вычисление выражения в текущем окружении
6. Вывод результата
7. Обновление окружения (при `define`)
8. Повторение цикла

2.4 Обработка ошибок

Все ошибки обрабатываются с использованием типа `Either String a`:

- Ошибки парсинга — сообщения от `Parsec`
- Ошибки выполнения — сообщения с описанием проблемы:
 - Неопределённая переменная
 - Ожидание числа
 - Деление на ноль
 - Переопределение встроенной константы
 - Некорректный аргумент для `car/cdr`

3 Структура исходного кода

Исходный код организован следующим образом:

```

1 import Data.Char (isAlphaNum, isDigit)
2 import Text.Parsec
3 import Text.Parsec.String (Parser)
4
5 data Val = Atom String | Number Integer | List [Val] | DottedPair
        Val Val
6     deriving (Show, Eq)
7
8 type Variables = [(String, Val)]
9
10 -- ===
11 whiteSpace :: Parser ()
12 atomParser :: Parser Val
13 numberParser :: Parser Val
14 emptyListParser :: Parser Val
15 dottedPairParser :: Parser Val
16 regularListParser :: Parser Val
17 listParser :: Parser Val
18 quotedParser :: Parser Val
19 parseExpr :: Parser Val
20 parseLisp :: String -> Either ParseError Val
21
22 -- ===
23 isForbiddenName :: String -> Bool
24 getVar :: String -> Variables -> Maybe Val
25 setVar :: String -> Val -> Variables -> Either String Variables
26
27 -- ===
28 toNumber :: Val -> Either String Integer
29 toBool :: Val -> Bool
30
31 -- ===
32 eval :: Variables -> Val -> Either String (Val, Variables)
33
34 -- === REPL ===
35 repl :: Variables -> IO ()
36 initialEnv :: Variables
37 main :: IO ()

```

3.1 Примечания по реализации

- Парсеры используют комбинатор `try` для реализации возврата при разборе альтернатив
- Пробелы игнорируются с помощью `whiteSpace`
- Окружение передаётся явно через аргумент функции `eval`
- Каждая операция возвращает новое окружение на случай изменений
- При бинарных операциях окружение последовательно обновляется при вычислении каждого аргумента